

Problématique :

Prédire la trajectoire d'un individu en se basant sur ses quelques derniers mouvements.

4.1. — Deterministic Scene-Aware Model

4.1.1. Data Processing

En utilisant les données du Dataset ETH/UCY déjà traité (https://drive.google.com/drive/folders/1REq_if6nqdw_jYtuRVPJqmDNTclxoJU?usp=sharing), se présentant sous la forme d'une liste d'images (frames) issues d'une vidéo de piéton et des bases de données qui vient du travail effectué sur le github suivant : <https://github.com/NasimShafiee/Introvert-Human-trajectory-prediction-via-conditional-3d-attention>

La première étape aura été de comprendre les données du dataset et l'architecture des bases de données.

Dans ce dataset, il y a pour chaque vidéo, un dossier contenant une image toutes les 10 frames de la vidéo, ainsi que 3 bases de données, chacune étant sous la même architecture :

	pos_x	pos_y	pos_x_delta	pos_y_delta	ped_id	frame_num	data_id
0	-1.57	5.57	-1.57	5.57	239	9920	245
1	-0.74	5.53	0.83	-0.04	239	9930	245
2	0.07	5.49	0.81	-0.04	239	9940	245
3	1.01	5.49	0.94	0.00	239	9950	245
4	1.95	5.41	0.94	-0.08	239	9960	245

La première BDD stock toutes les données, ensuite cet ensemble a été divisé en 2, une partie pour l'entraînement des modèles et l'autre pour leur évaluation. Cela permet de tester nos modèles sur des données qu'ils ne se sont pas entraînés dessus.

Le but a été ensuite de pouvoir travailler avec ces données, cependant il s'agit de coordonnées du monde et pas en pixels et donc il faut utiliser une matrice homographique. Fort heureusement, elles ont déjà été calculer et disponible sur : <https://github.com/CORE-SNU/ECP-MPC/tree/main/assets/homographies>

Avec celle-ci, on est capable d'extraire ces coordonnées sur l'images et de retracer la position ou trajectoire complète d'un ou plusieurs piétons.



4.1.2. — Baseline Model

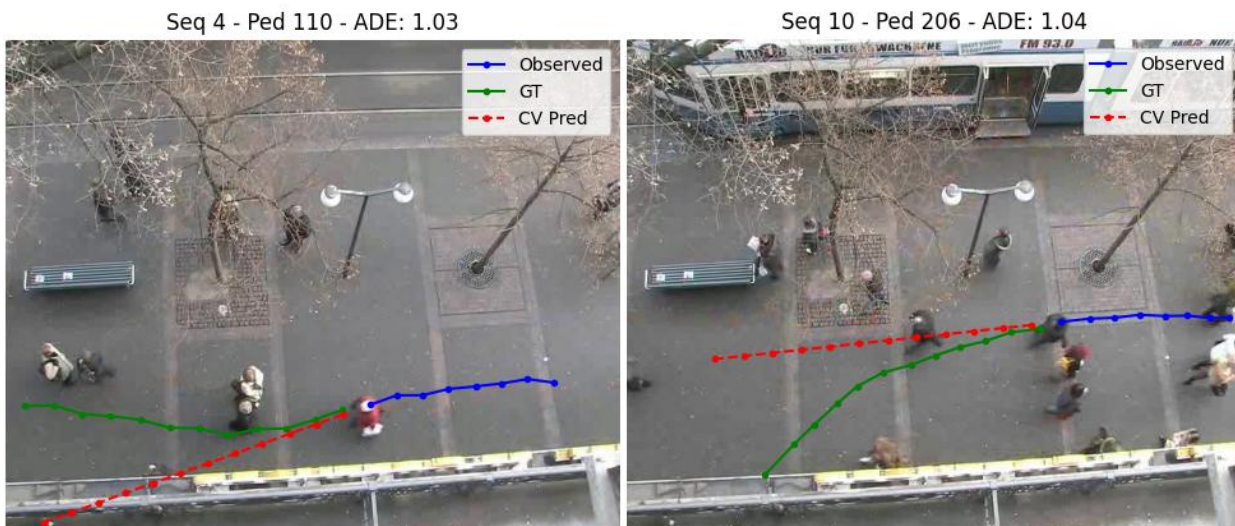
Avant de commencer à développer et entrainer des modèles d'IA sur notre problématique qu'est la prédiction de trajectoire de piéton, il est en parti possible de prédire la trajectoire d'un piéton via une extrapolation à vitesse constante.

En récupérant les derniers pas effectué par un piéton, s'il on applique le même vecteur, on peut prétendre prédire l'avenir.



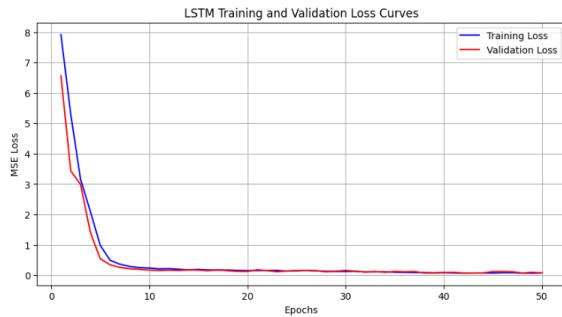
Cela marche étonnamment bien lorsqu'il s'agit d'une trajectoire en ligne droite ou statique, ce qui est dans la plupart du temps le cas pour un piéton lambda.

Cependant, il arrive parfois que cela ne marche pas comme prévu :



Dans les cas d'un changement de direction ou d'obstacle, cette méthode ne peut pas suivre le choix humain et l'environnement du piéton de même que les interactions sociales car souvent, le piéton n'est pas seul dans la scène.

4.1.3. LSTM Encoder-Decoder



On peut observer que dans le cas d'un LSTM simple (encoder, decoder) on a pour les premières epochs une décroissance rapide de l'erreur quadratique moyenne et se perfectionne par la suite.

L'utilisation du MSE permet d'atteindre une moyenne des trajectoires envisageable en pénalisant grandement un écart entre la vraie trajectoire future et la prédiction. Si par exemple si on prédit que le piéton va tourner à gauche alors qu'il allait tout droit, cela va impacter énormément la fonction de perte et le modèle va apprendre que dans ce cas-là, il est préférable d'aller tout droit.

Et au fil des epochs, le modèle en se perfectionnant fait plus attention aux petits détails et peut ainsi prédire à partir d'un petit changement de direction, le comportement à suivre dans cette situation.

Cependant, on ne peut pas prétendre deviner le comportement humain, et les changements de direction soudain, ou esquive d'autres piétons aura tendance à être assouplis lors de la prédiction car prédire un tel changement impactera beaucoup trop la MSE et donc le modèle prédit une courbure plus souple afin de réduire les chances de se tromper et d'être davantage pénaliser.



Comme on peut le voir ici, avec la comparaison de la prédiction à vitesse constante, le LSTM tends plus à suivre la trajectoire le plus souvent emprunté au lieu de foncé dans le mur.

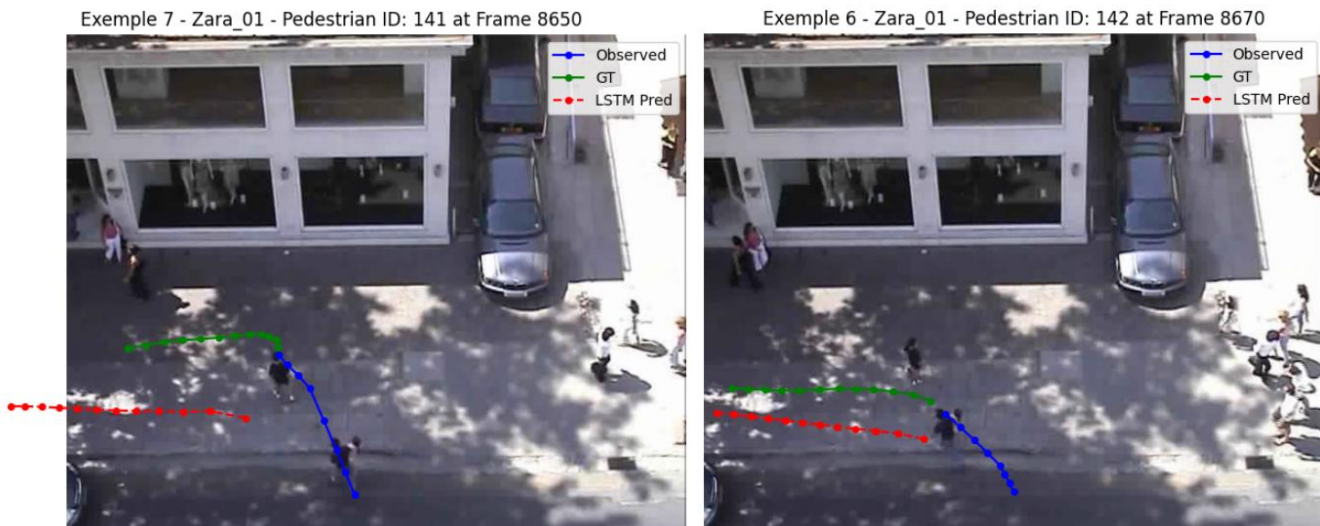
Cependant, il n'y a pas de prédiction des autres piétons et le modèle ne comprends donc pas qu'il faut esquiver un groupe de personne. Cependant il apprend à ne pas rentrer dans les murs ou arbre, ban, etc en se basant sur son entraînement qui lui a appris les chemin 'parcourable'. Pour approfondir d'avantage ce sujet d'interaction social du modèle, on pourrait se baser sur le travail suivant :

https://cvgl.stanford.edu/papers/CVPR16_Social_LSTM.pdf qui porte sur un Social LSTM pour la prédiction de trajectoire humaine dans un espace bondé. Mais dans le temps imparti du projet, je n'ai pas eu le temps de tester celui-ci.

Au final, comparé à la prédiction à vitesse constante, le LSTM standard propose une nette amélioration de plus de 50% pour la ADE et la FDE sur la scène de l'hôtel :

```
--- Evaluation and Comparison ---  
LSTM Model    - ADE: 0.2504, FDE: 0.4447  
CV Baseline   - ADE: 0.5532, FDE: 1.1891  
  
Improvement: ADE: 54.73%, FDE: 62.60%
```

Modèle / visualisation bug :

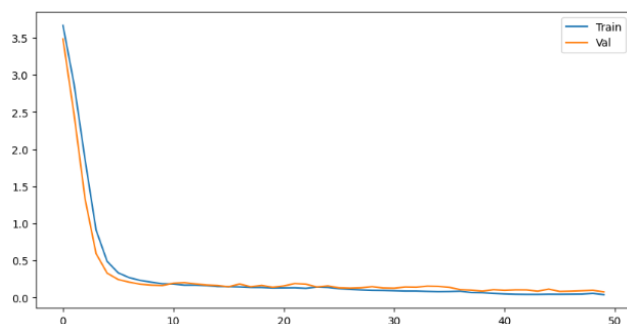


En essayant de ré-entraîner le modèle sur les données d'une autre scène, je suis tombé sur 2 cas assez spécifique me donnant lors de la visualisation des prédictions qui semble commencé à un certain offset alors que tous les autres piétons de cette scène donnent des prédictions stable et cohérents, ces deux individus restent un mystère dont je n'ai pas eu le temps de résoudre.

4.1.4. Scene-Aware LSTM (Vision + Motion)

Afin d'améliorer le modèle LSTM, il nous est possible d'y inclure un CNN pré-entraîné sur des données visuelles afin d'ajouter une reconnaissance d'image à la scène, permettant de reconnaître un ban, un arbre ou une voiture garée.

Pour tester cette hypothèse, on inclus un ResNet-18 pour extraire les caractéristiques visuelles de la scène et un LSTM pour garder le côté temporelle du déplacement du piéton. On prédit ainsi la trajectoire future en fusionnant ces deux représentations avant de les passer dans le decoder.



```
--- Multimodal Model Evaluation Results ---  
Multimodal ADE: 0.2475, FDE: 0.3566  
LSTM-only ADE: 0.2504, FDE: 0.4447  
  
Improvement over LSTM-only: ADE: 1.17%, FDE: 19.82%
```

On a donc une courbe d'apprentissage similaire au LSTM basic, mais l'évaluation du modèle montre une légère amélioration des métriques de mesures ADE et FDE allant jusqu'à 20% d'amélioration par rapport au LSTM simple.

4.2. — GAN for Multimodal Prediction

4.2.1. — GAN Theory Review

Le GAN ou Generative Adversarial Network est une architecture de machine learning génératif ayant pour objectif de créer de nouvelles données au plus proche de la réalité.

Il peut s'agir de générer une image, du texte ou de l'audio, à partir d'un jeu de données déjà existant.

Il est composé de 2 réseaux de neurones :

- **Le Générateur**

Le générateur a pour rôle de créer des données qui doivent ressembler au mieux celles du dataset, dans notre cas il aura pour mission de générer une trajectoire future ressemblant au mieux à la réalité.

Il commence à partir d'un bruit aléatoire et des informations de la scène comme la trajectoire passée ou une image de la scène.

- **Le Discriminateur**

Le discriminateur a pour rôle de deviner si ce qu'on lui montre est issue du vrai dataset ou si c'est des données totalement créées par le générateur.

Pour cela, si on prend notre exemple, on lui montre aléatoirement la trajectoire réelle ou la trajectoire « fausse » générées par le générateur.

Il doit donner une note entre 1 s'il pense que c'est réel ou 0 s'il devine que c'est faux.

En combinant ces deux réseaux de neurones, on les entraîne simultanément, soit à déceler la vérité soit à tromper au mieux le discriminateur. De cette manière, le générateur apprend à générer des données toujours plus proches de la réalité.

Cependant, cela peut amener à quelques limites du modèle génératif dans le cas où le générateur trouve un cheat code qui trompe trop bien le discriminateur à un instant T .

Le générateur ayant trouvé une stratégie viable, continue à l'exploiter au point de ne produire uniquement la même donnée peu importe le bruit initial.

On appelle ce phénomène le **mode collapse** ou l'effondrement du mode.

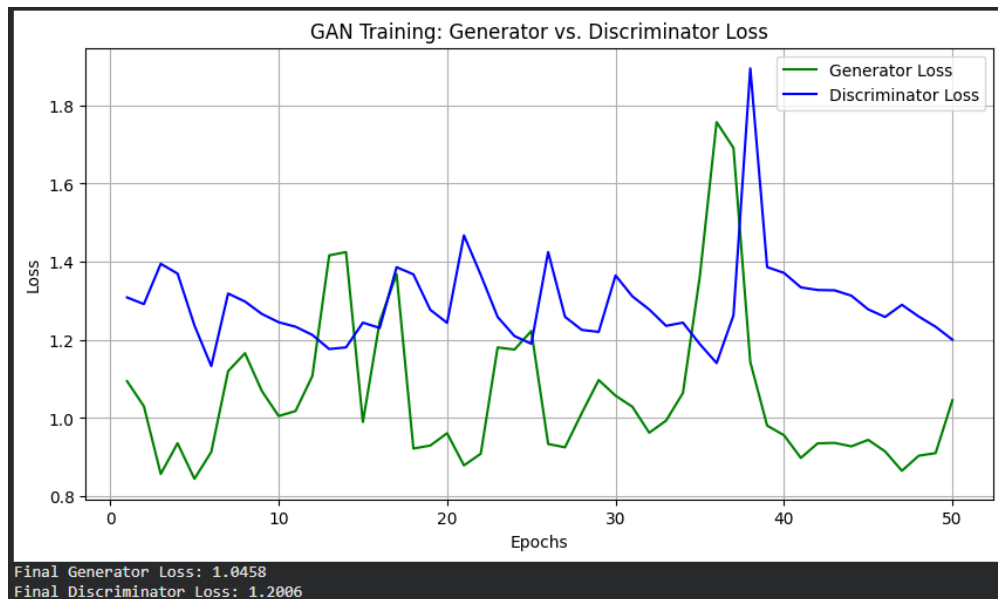
Les GANs comparés à un modèle LSTM simple, ne sont pas limités par l'erreur quadratique moyenne qui force le modèle à viser une moyenne donnant des trajectoires floues, fluides ou irréalistes. Les GANs cherchent à être réalistes.

On peut dire que **les GANs modélisent la multimodalité** car pour une même trajectoire passée, s'il on explore les différentes possibilités en variant le bruit, on crée plusieurs trajectoires « réelles ».

Par exemple, si avec z_1 on prédit tourné à gauche et avec z_2 on prédit de continuer tout droit, les deux sont des possibilités « réelles » qui peuvent très bien exister toutes les deux dans la vraie vie. Alors que s'il on utilisait un LSTM avec une MSE loss, il aurait fait une moyenne des deux en fonction de la répartition de son entraînement.

4.2.2. — Vanilla Trajectory GAN

Le but ici était d'implémenter un GAN pour la prédiction de la trajectoire des piétons.



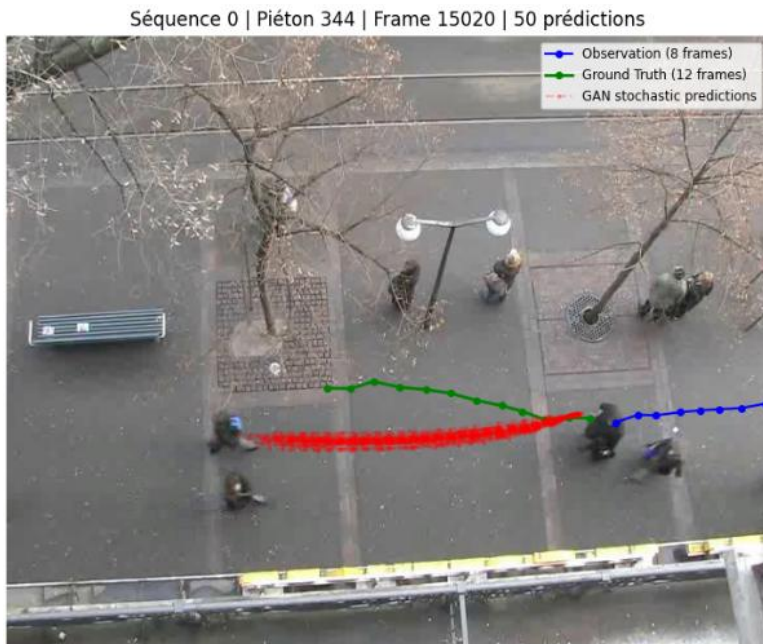
On peut observer durant l'entraînement que le générateur et le discriminateur se batte à tour de rôle pour tromper l'autre ou faire valoir la vérité.

On perçoit également un pic à l'entour de l'epoch 38 qui pourrait s'expliquer soit par une tentative de sortir d'un minimum local, soit d'une instabilité qui aurait été corrigé par le gradient clipping.

Cependant, bien que ces courbes d'apprentissage, les résultats montrent qu'un mode colapse à eu lieu et que les prédictions effectuées par le générateur sont toutes les mêmes :

--- Analyse Best-of-10 ---				
	Sample	ADE (m)	FDE (m)	Max Error (m)
4	Prediction 5	3.7616	7.8772	7.8772
3	Prediction 4	3.7619	7.8775	7.8775
9	Prediction 10	3.7619	7.8771	7.8771
8	Prediction 9	3.7619	7.8757	7.8757
6	Prediction 7	3.7620	7.8770	7.8770
2	Prediction 3	3.7620	7.8767	7.8767
7	Prediction 8	3.7621	7.8775	7.8775
0	Prediction 1	3.7622	7.8763	7.8763
5	Prediction 6	3.7622	7.8778	7.8778
1	Prediction 2	3.7624	7.8771	7.8771

Et malgré les essais en essayant de réduire l'impact de la MSE jusqu'à qu'elle soit nulle, d'amplifier l'impact du bruit en doublant sa dimension, en essayant de rééquilibrer le combat en divisant par 10 le taux d'apprentissage du discriminateur ou même en descendant le label smothing, rien n'a sauvé mon modèle qui resta coincé en mode colapse.



Chaque prédiction se ressemblant toutes, la même courbure malgré un bruit différent.

J'ai donc demandé de l'aide à un ami, malgré que tout ce travail fût réalisé seul, Zhongtian avait réussi à créer un GAN qui n'était pas tombé dans un mode colapse.

En se basant sur son architecture, adapté à ma logique de code et d'extraction des données, un nouveau GAN est né et propose des résultats promettants.



On peut voir ici la diversité des futures possible avec dans certains de ceux-ci une ou plusieurs prédictions qui semble très proche de la vraie trajectoire.

On peut ainsi affirmer que le GAN peut produire différents futures possibles.

Chaque future prédit étant légèrement des autres montrant ainsi que le GAN n'est pas un modèle déterministe.

Sur l'ensemble des prédictions faites sur le dataset val de l'hôtel, on observe une diversité moyenne de 0.2m ce qui montre à la fois la proximité des prédictions mais ainsi qu'on n'obtient pas à chaque fois la même prédiction.

Score de Diversité Moyen : 0.2436 mètres

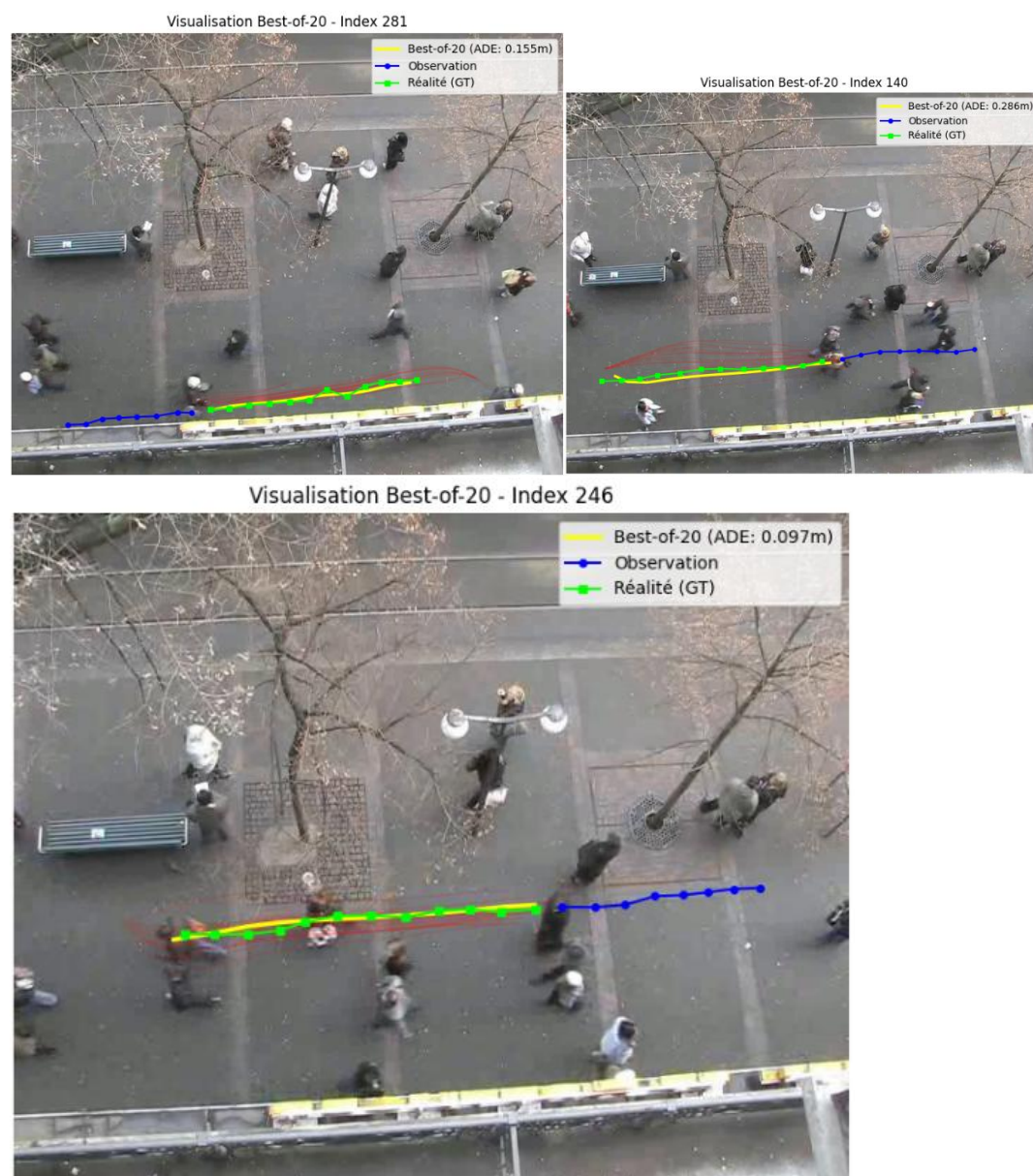
4.2.3. - Best-of-K Sampling

Cependant une question reste, est-ce que cet ensemble de prédiction est proche de la réalité, c'est ce pourquoi on calcule une moyenne de l'ADE et FDE et on obtient un score moyen meilleur que tous les modèles précédents :

```
=====
SCORES FINAUX SUR POS_DATA_VAL (K=20)
=====
L'ADE@20 moyen est de : 0.1844 mètres
Le FDE@20 moyen est de : 0.3089 mètres
```

Cela montre ainsi que vouloir prédire un avenir certain est moins efficace qu'un ensemble de future proche de la réalité.

Il y aura toujours un futur prédit proche de la réalité :



Code et résultats présent sur le github suivant :

<https://github.com/DylanQuellet/Pedestrian-Trajectory-Prediction>